

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



PRINCIPLES OF DATABASE MANAGEMENT
IT079IU

FINAL REPORT

Topic: Online Voting System

BY GROUP 05 – MEMBER LIST

1	Phạm Thái Thục Minh	ITDSIU21022	Team Leader
2	Đỗ Thanh Bảo Anh	ITDSIU22175	Team Member
3	Đào Ngọc Lan Hồng	ITDSIU21088	Team Member
4	Phạm Tuấn Đăng Khoa	ITITI22087	Team Member
5	Lâm Quang Lợi	ITCSIU21084	Team Member
6	Nguyễn Thị Ngọc Mai	ITDSIU21098	Team Member
7	Trần Triệu Như	ITDSIU21029	Team Member

Instructor: Assoc. Prof. Nguyen Thi Thuy Loan

Table Contents

List of Figure.....	4
List of Table	5
CHAPTER I. INTRODUCTION.....	6
1. Abstract.....	6
2. System overview	6
3. Goal	6
4. The techniques & tools used.....	7
CHAPTER II. TASK TIMELINE & DIVISION	8
1. Contribution.....	8
2. Project timeline & task division.....	8
CHAPTER III. THEORETICAL BASIS.....	11
1. ERD - EERD(Entity - Relationship Diagram - Enhanced Entity-Relationship model)	11
2. Convert ERD model to Relational model	11
3. Normalization.....	11
CHAPTER IV. PROJECT ANALYSIS	12
1. Requirements Analysis	12
2. Approach Analysis	12
2.1 Reviewed Materials	12
2.2 Research Approach.	12
3. System Analysis.....	13
3.1 Database Design.....	13
3.2 Convert ERD model to relational model	15
3.3 Normalization	15
3.4 Create database in SQL Server	16
3.5 Collect and insert value in database	17
CHAPTER V. CONNECTION BETWEEN SQL AND JAVA.....	19
1. SQL Syntax.....	19
2. Relational Algebra, Tree, SQL Query	19
3. Application forms.....	31
3.1 Set Up Development Environment.....	31
3.2 Create Database Schema	31
3.3 Database Connection in Java and Create the Application Form Using Swing.....	31
4. Demo	34
4.1 Admin	34
4.2 Voter	37
4.3 Election Result	38

CHAPTER VI. CONCLUSION	40
1. Conclusion	40
2. Recommendation.....	40
3. What we learn after project ?	40
Reference:	41

List of Figure

Figure 3.1.1 Entity Relationship Diagram using Non-Binary Relationships	13
Figure 3.1.2 Entity Relationship Diagram Plus using Binary Form Relationships.....	14
Figure 3.1.3 Schema Diagram for the Online Voting System	14
Figure 3.1.4 Database Diagram for the Online Voting System	15
Figure 3.4.1 Create table Admin	16
Figure 3.4.2 Create table Voter	16
Figure 3.4.3 Create table Candidate.....	16
Figure 3.4.4 Create table Election.....	16
Figure 3.4.5 Create table votes	17
Figure 3.5.1 Insert value into table Admin.....	17
Figure 3.5.2 Insert value into table Voter	17
Figure 3.5.3 Insert value into table Candidate	17
Figure 3.5.4 Insert value into table Election	18
Figure 3.5.5 Insert value into table votes.....	18
Figure 4.3.1 Voter login code	32
Figure 4.3.2 Successful Voter login	33
Figure 4.3.3 Failed Voter login	33
Figure 4.4.1 Admin sign-up interface	34
Figure 4.4.2 Admin manage Voters.....	34
Figure 4.4.3 Add Election	35
Figure 4.4.4 Add Candidate.....	35
Figure 4.4.5 Manage Voters.....	36
Figure 4.4.6 Voter sign-up interface.....	37
Figure 4.4.7 Voter select Election	37
Figure 4.4.8 Cast your vote.....	38

List of Table

Table 2.1 Individual responsibility and contribution..... 8

Table 2.2 Planning and task division for individuals and teams 8

CHAPTER I. INTRODUCTION

1. Abstract

In modern society, voting remains important in a variety of situations, from organizational decisions to public elections. Traditional voting methods often face challenges such as inefficiency and limited accessibility. Therefore, developing an online voting system is necessary to solve these problems, bringing convenience and efficiency. However, the use of external online voting systems can pose security risks and may not meet specific database requirements, potentially affecting the voting process in a university environment.

This project aims to apply principles from Database Management and Adaptive Technologies to create a secure and efficient online voting system for organizations in academic environments such as unions and departments. By leveraging database design techniques and modern technologies, the project seeks to improve the accuracy and efficiency of the voting process, making it more accessible and time-saving.

This approach includes the use of Entity Relationship Diagrams (ERD), schema design, and normalization techniques to optimize the database structure. From there, we facilitate the creation of a simple and user-friendly interface, especially for students. The findings show that structured databases and intuitive user interfaces significantly enhance the effectiveness of voting systems.

In summary, developing an internal online voting system based on these principles provides a viable solution to the limitations of traditional voting methods, ensuring a safe, accurate, and effective voting process in the digital age. We hope this research will provide insights into data design, connecting data to applications, and implementing voting technology in university settings. Furthermore, we hope that this project will become a simple and easy-to-use solution for voting for key positions in the learning environment and pave the way for future advances in online voting technology.

2. System overview

The deployment of the Online Voting System marks a significant step in modernizing the electoral process. This system optimizes elections by integrating functionalities like voter registration, secure voting, and real-time result computation. Voters and administrators log in using unique credentials, with administrators managing voter registration for security. Administrators also configure elections, set timings, and manage candidate lists. Voters cast their votes through a secure interface that ensures the "one person, one vote" principle.

Post-voting, the system calculates and displays results, accessible to all voters. Security measures include unique login IDs, passwords, and secure data storage and transmission. This overview highlights the system's role in streamlining and safeguarding the election process.

3. Goal

The goal of the Online Voting System is to modernize and streamline the electoral process by providing a secure, efficient, and user-friendly platform for voter

registration, voting, and result computation. The system aims to enhance accessibility for all eligible voters, ensure the integrity of each vote through robust security measures, and simplify the management of elections for administrators. By transitioning from traditional paper ballots to a digital format, the system seeks to reduce costs, minimize administrative workload, thereby efficiency of the electoral process

4. The techniques & tools used

- Java language uses IntelliJ IDEA Community Edition and Visual Studio Code to design the user interface from the database, application logic flow, and process back-end.
- SQL language uses Microsoft SQL Server Management Studio to create user databases and process logical queries.
- Use Draw.io and ERDPlus to draw entity relationship diagrams and schema.

CHAPTER II. TASK TIMELINE & DIVISION

1. Contribution

Table 2.1 Individual responsibility and contribution

Name	Responsibility	Contribution
Phạm Thái Thực Minh	ERD Specialist, Proposal Writer, Database Integrator, Data Optimizer, Quality Assurance, Final Reviewer, Presenter	14.286%
Đỗ Thanh Bảo Anh	ERD Analyst, Schema Planner, Back-end Engineer, Data Normalizer, Query Developer, Quality Analyst, Final Reviewer	14.286%
Đào Ngọc Lan Hồng	ERD Analyst, Schema Planner, Database Developer, Front-end Engineer, Quality Analyst, Slide Creator	14.286%
Phạm Tuấn Đăng Khoa	Front-end Engineer, Back-end Engineer, Application Developer, Quality Analyst, Presenter	14.286%
Lâm Quang Lợi	Proposal Author, Back-end Engineer, Application Developer, UI Integrator, Presenter	14.286%
Nguyễn Thị Ngọc Mai	ERD Analyst, Database Developer, Query Developer, Front-end Engineer, Presenter	14.286%
Trần Triệu Như	Proposal Author, Schema Planner, Database Developer, Front-end Engineer, UI Integrator, Presenter	14.286%

2. Project timeline & task division

Table 2.2 Planning and task division for individuals and teams

Stage	Week	Tasks	Responsible Member(s)
Planning and Design	1	Selecting the group topic: Online Voting System	All Members
	2	Drafting the proposal report: outline and initial draft	Triệu Như, Quang Lợi, Thực Minh
	3	Drafting the proposal report: review and finalization	Thực Minh

	3	Research existing online voting systems: functionality and technology stack	All Members
	3	Planning system structure and deciding tools and technologies: meeting and brainstorming	All Members
Conceptual Data Modeling	4	Research on entity relationship diagrams (ERD)	All Members
	4,5	Design ERD: initial draft and iterations	Ngọc Mai, Bảo Anh, Thục Minh
	5	Design database schema: tables, attributes, and relationships	Lan Hồng, Ngọc Mai, Triệu Như
	5,6	Define keys (Primary/Foreign) for each table	Ngọc Mai, Lan Hồng, Triệu Như
	6	Prepare Midterm Report: compile and organize findings	Bảo Anh, Ngọc Mai, Lan Hồng
Data Collection and Normalization	6	Setup SQL Server database: installation and configuration	All Members
	6,7	Create and normalize tables with SQL Server: initial design and refinement	Ngọc Mai, Lan Hồng, Triệu Như, Bảo Anh, Thục Minh
	7	Data normalization: eliminate redundancy and ensure consistency	Thục Minh, Bảo Anh
Query Design	7,8	Design and optimize SQL queries for data retrieval and manipulation	Thục Minh, Bảo Anh
Application Development	6,7,8	Design and develop the overall user interface: wireframing and mockups (Java)	Đặng Khoa, Quang Lợi
	7,8	Develop the user interface (Java): initial coding	Đặng Khoa, Quang Lợi
	8	Integrate the user interface with the back-end (Java): connect API endpoints	Quang Lợi, Đặng Khoa

Database Connection	7,8	Connect back-end server (Java) and SQL Server database: configuration and testing	All Members
	8,9,10	Implement back-end server (Java): coding and API development	Đặng Khoa, Bảo Anh, Quang Lợi
	9,10	Integrate back-end with front-end (Java): connect UI with back-end services	Triệu Như , Ngọc Mai, Đặng Khoa
Front-end Development	8, 9	Develop the user interface (Java): coding and styling	Triệu Như , Lan Hồng, Ngọc Mai
	9, 10	Test and debug the user interface: functionality and performance testing	Triệu Như , Lan Hồng
Testing	11	Create and execute test cases: unit and integration testing	Đặng Khoa, Bảo Anh, Quang Lợi
	11	Review, improve code, and resolve bugs: code reviews, refactoring, tracking, and fixing bugs	Đặng Khoa, Quang Lợi
Report and Presentation	12	Final report: compiling and writing final documentation	All Members
	12,13	Create slides: design and organize presentation slides	Lan Hồng
	13	Overall checking: review final project and presentation	Thục Minh, Bảo Anh

CHAPTER III. THEORETICAL BASIS

1. ERD - EERD(Entity - Relationship Diagram - Enhanced Entity-Relationship model)

Definition

An Entity Relationship Diagram (ERD) is a type of flowchart that illustrates how “entities” such as people, objects or concepts relate to each other within a system. They mirror grammatical structure, with entities as nouns and relationships as verbs.

Enhanced entity-relationship (EERD) models, also known as extended entity-relationship models, are advanced database diagrams very similar to regular ER diagrams.

2. Convert ERD model to Relational model

Step 1: Convert Regular or Strong Entity

Step 2: Convert Weak Entity Types

Step 3: Mapping Binary 1-to-1

Step 4: Mapping Binary 1-to-N

Step 5: Mapping Binary M-to-N

Step 6: Convert Multivalued Attributes

Step 7: Specialization/Generalization

3. Normalization

1NF (First Normal Form): A table is in 1NF if every cell contains only atomic (indivisible) values, and each record needs to be unique.

2NF (Second Normal Form): A table is in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.

3NF (Third Normal Form): A table is in 3NF if it is in 2NF and there are no transitive dependencies of non-key attributes on the primary key.

BCNF (Boyce-Codd Normal Form): A table is in BCNF if it is in 3NF and for every one of its dependencies $X \rightarrow Y$, X is a super-key. In other words, the only determinant of a non-trivial functional dependency is a candidate key.

CHAPTER IV. PROJECT ANALYSIS

1. Requirements Analysis

The system should be able to meet the following standards based on the supplied instructions regarding our selected topic. The requirements for the system are defined as the project's establishment:

- The system needs to provide users with functions such as login using username and password, specifying whether the current user is a admin or voter.
- After logging in, the system will restrict users' interaction based on their role in the election. Candidates can view the election, while voters can vote for their preferred candidate, following the principle of 'one person, one vote'.
- A user with an administrator role can create the election (setting up election time, candidate position, and candidates).
- Vote counting will be handled by the system once the voting period concludes. If there is a need to recount, the administrator should have access to the database because the voter's participation in various elections is accurately recorded and managed.
- After the vote counting is complete, everybody should be allowed to view the result in "Election". However, if the voting process is on-going, the result of the current state should not be viewable, even by the administrator, to prevent any potential fraud.
- The system consists of data related to students, with each student considered a user of the system. The data contained in the database includes students' basic information such as full name, gender, age, email, and their username and password. Advanced encryption and data protection measures should be incorporated to safeguard personal information against unauthorized access and breaches.

2. Approach Analysis

2.1 Reviewed Materials

- Hoffer, J., Ramesh, V. and Topi, H. (2016) Modern Database Management. Harlow, United Kingdom: Pearson Education Limited.
- Tech, A. (2021) Connect SQL server with visual studio 2022, YouTube. Available at: <https://www.youtube.com/watch?v=HeGNMgdQ79M> (Accessed: 24 May 2024).
- Alexander, H. (2024) Connect to SQL server in Java - Guide for Beginners, Devart Blog. Available at: <https://blog.devart.com/connect-to-sql-server-in-java.html> (Accessed: 24 May 2024).

2.2 Research Approach.

The online voting system, which aims to provide secure and efficient voting, follows an agile methodology. The team sets all necessary features, including voter and candidate management, election setup, secure voting, vote counting, admin tasks, and data confidentiality by using SQL Server, JDBC, and Java.

3. System Analysis

3.1 Database Design

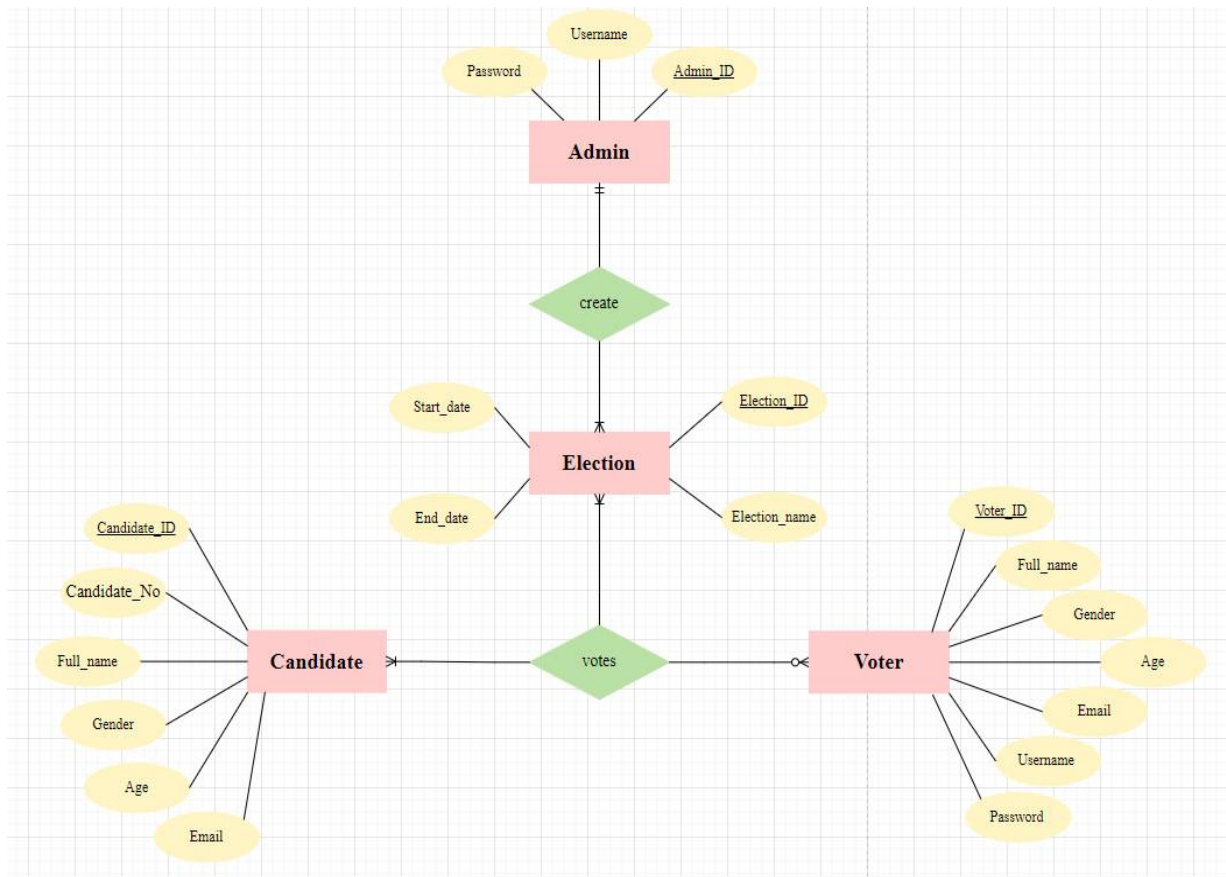


Figure 3.1.1 Entity Relationship Diagram using Non-Binary Relationships

Converting Non-Binary Relationships to Binary Form

In general, any non-binary relationship can be represented using binary relationships by creating an artificial entity set.

Replace 'votes' between entity sets 'Election', 'Voter' and 'Candidate' by an entity set 'votes', and three relationship sets:

1. 'be voted', relating 'votes' and 'Candidate'
2. 'cast', relating 'votes' and 'Voter'
3. 'determine', relating 'votes' and 'Election'

So, the new entity 'votes' is identified by the composite key for which each attribute that makes up the key is a foreign key in its own right.

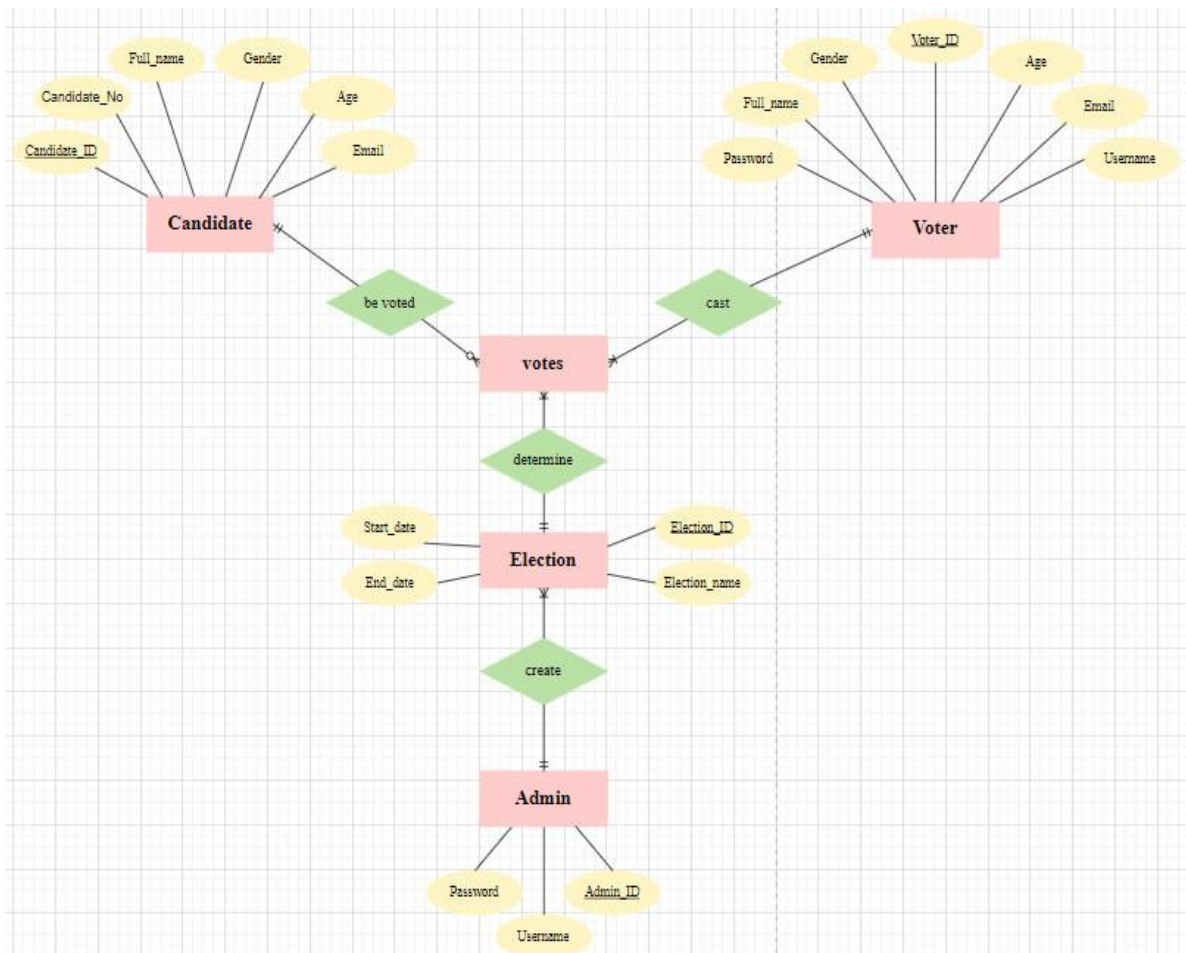


Figure 3.1.2 Entity Relationship Diagram Plus using Binary Form Relationships

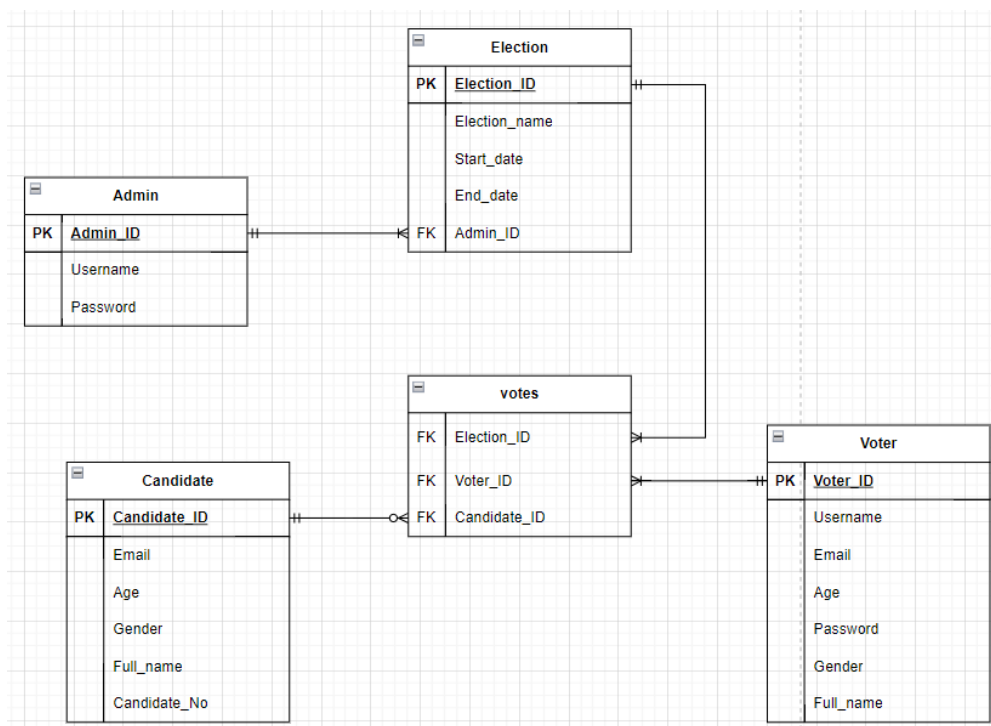


Figure 3.1.3 Schema Diagram for the Online Voting System

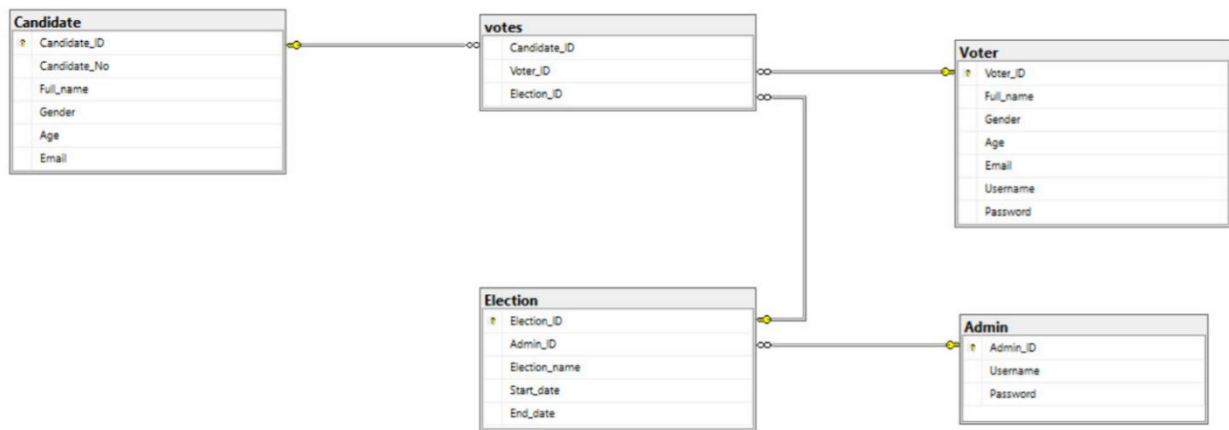


Figure 3.1.4 Database Diagram for the Online Voting System

3.2 Convert ERD model to relational model

- Admin (Admin_ID, Username, Password)
- Candidate (Candidate_ID, Candidate_No, Full_name, Gender, Age, Email)
- Voter (Voter_ID, Full_name, Gender, Age, Email, Username, Password)
- Election (Election_ID, Election_name, Start_date, End_date, Admin_ID)
- votes (Election_ID, Candidate_ID, Voter_ID)

3.3 Normalization

First Normal Form (1NF): Each table has a primary key: 'Admin' has 'Username', 'Candidate' has 'Candidate_ID', 'Voter' has 'Voter_ID', 'Election' has 'Election_ID', and 'votes' has a composite key ('Election_ID', 'Candidate_ID', 'Voter_ID'). All other attributes in each table are dependent on the primary key.

Second Normal Form (2NF): There are no partial dependencies in the tables. Each non-key attribute is fully functionally dependent on the entire primary key.

Third Normal Form (3NF): There are no transitive dependencies in the tables. This means that each non-key attribute is directly dependent on the primary key, not on other non-key attributes.

Boyce-Codd Normal Form (BCNF): All the tables - Admin, Candidate, Voter, Election, and votes - adhere to the BCNF as the left-hand side of each functional dependency is a candidate key. This means that there are no non-trivial functional dependencies of attributes on anything other than a super-set of a candidate key.

3.4 Create database in SQL Server

```
Create Database [Online-Voting]
GO
Use [Online-Voting]
Go

Create table Admin (
Admin_ID varchar(50) not null primary key,
Username varchar(50) not null,
Password varchar(50) not null
)
Go
```

Figure 3.4.1 Create table Admin

```
Create table Voter (
Voter_ID varchar(50) not null primary key,
Full_name Nvarchar(50) not null,
Gender varchar(50) not null,
Age int not null,
Email varchar(50) not null,
Username varchar(50) unique,
Password varchar(50) not null
)
Go
```

Figure 3.4.2 Create table Voter

```
Create table Candidate (
Candidate_ID varchar(50) not null primary key,
Candidate_No int,
Full_name Nvarchar(50) not null,
Gender varchar(50) not null,
Age int not null,
Email varchar(50) not null,
)
Go
```

Figure 3.4.3 Create table Candidate

```
Create table Election (
Election_ID varchar(50) PRIMARY KEY,
Admin_ID varchar(50),
Election_name VARCHAR(50) NOT NULL,
Start_date SMALLDATETIME,
End_date SMALLDATETIME,
FOREIGN KEY(Admin_ID)REFERENCES Admin (Admin_ID)
)
Go
```

Figure 3.4.4 Create table Election

```

Create table votes(
Candidate_ID varchar(50),
Voter_ID varchar(50),
Election_ID varchar(50),
FOREIGN KEY(Election_ID)REFERENCES Election (Election_ID),
FOREIGN KEY(Voter_ID)REFERENCES Voter (Voter_ID),
FOREIGN KEY(Candidate_ID)REFERENCES Candidate (Candidate_ID)
)
Go

```

Figure 3.4.5 Create table votes

Entities:

Admin: Represents the individuals responsible for managing and configuring the online voting system. It may have attributes such as Admin_ID, Username, and Password.

Candidate: Represents the individuals running for election within the online voting system. It may have attributes such as Candidate_ID, Candidate_No, Full_name, Gender, Age, and Email.

Voter: Represents the individuals who have the right to cast their votes in the online voting process. It may have attributes such as Voter_ID, Full_name, Gender, Age, Email, Username, and Password.

Election: Represents the electoral events with defined timelines and procedures, overseen by administrators, to ensure a fair voting process. It may have attributes such as Election_ID, Election_name, Start_date, and End_date.

Relationship:

"votes" Relationship: A many-to-many relationship between the Candidate, Voter, and Election entities. This indicates that voters can cast their votes for multiple candidates in different elections, and candidates can receive votes from multiple voters in various elections.

3.5 Collect and insert value in database

```

Insert into Admin (Admin_ID, Username, Password)
values ('A01', 'Admin', 'jojo')

```

Figure 3.5.1 Insert value into table Admin

```

Insert into Voter (Voter_ID, Full_name, Gender, Age, Email, Username, Password)
values ('ITDSIU22161', N'Phạm Hồng An', 'Female', 20, 'ITDSIU22161@hcmiu.edu.vn', 'pha22161', '2ItZTtjs'),
('ITDSIU22135', N'Dương Ngọc Phương Anh', 'Female', 20, 'ITDSIU22135@hcmiu.edu.vn', 'dnpa22135', '5om1CCGy'),
('ITDSIU21075', N'Nguyễn Ngọc Lan Anh', 'Female', 20, 'ITDSIU21075@hcmiu.edu.vn', 'nnla21075', 'OS5vEPk'),
('ITDSIU22177', N'Nguyễn Xuân Trâm Anh', 'Female', 20, 'ITDSIU22177@hcmiu.edu.vn', 'nxta22177', 'W4mpFRBu'),
('ITDSIU22147', N'Phạm Nguyễn Phan Anh', 'Female', 20, 'ITDSIU22147@hcmiu.edu.vn', 'pnpa22147', 'AC8YfU66'),
('ITDSIU22130', N'Phạm Nguyễn Quỳnh Anh', 'Female', 20, 'ITDSIU22130@hcmiu.edu.vn', 'pnqa22130', 'G5p98n4D'),
('ITDSIU21073', N'Phạm Vũ Tuyết Anh', 'Female', 21, 'ITDSIU21073@hcmiu.edu.vn', 'pvta21073', 'VTqYV0rJ'),
('ITDSIU21001', N'Phan Quốc Anh', 'Male', 21, 'ITDSIU21001@hcmiu.edu.vn', 'pqa21001', '9tkzJKqN'),
('ITDSIU22151', N'Nguyễn Hoàng Hồng Ân', 'Male', 20, 'ITDSIU22151@hcmiu.edu.vn', 'nhha22151', 'qgKv54qR'),

```

Figure 3.5.2 Insert value into table Voter

```

Insert into Candidate(Candidate_ID, Candidate_No, Full_name, Gender, Age, Email)
values
('C01', 1, N'Đào Hữu Huy', 'Male', 20, 'ITCSIU22348@hcmiu.edu.vn'),
('C02', 2, N'Trần Triệu Như', 'Female', 20, 'ITITI22614@hcmiu.edu.vn'),
('C03', 3, N'Phạm Tuấn Anh Khoa', 'Male', 21, 'ITDSIU21850@hcmiu.edu.vn'),
('C04', 4, N'Nguyễn Hoàng Bảo Ngân', 'Female', 24, 'ITCSIU19040@hcmiu.edu.vn'),
('C05', 5, N'Trương Trí Dũng', 'Male', 20, 'ITITI22836@hcmiu.edu.vn')

```

Figure 3.5.3 Insert value into table Candidate

```
Insert into Election (Election_ID, Election_name, Start_date, End_date, Admin_ID)
values('SP2024S2', 'Student president', '2024-01-01 7:00:00', '2024-04-30 23:59:59', 'A01')
```

Figure 3.5.4 Insert value into table Election

```
Insert into votes (Election_ID, Voter_ID, Candidate_ID)
values('SP2024S2', 'ITDSIU22161', 'C01'),
('SP2024S2', 'ITDSIU22135', 'C02'),
('SP2024S2', 'ITDSIU21075', 'C03'),
('SP2024S2', 'ITDSIU22177', 'C04'),
('SP2024S2', 'ITDSIU22147', 'C05'),
('SP2024S2', 'ITDSIU22130', 'C05'),
('SP2024S2', 'ITDSIU21073', 'C03'),
('SP2024S2', 'ITDSIU21001', 'C05'),
('SP2024S2', 'ITDSIU22151', 'C01'),
('SP2024S2', 'ITDSIU22157', 'C02'),
```

Figure 3.5.5 Insert value into table votes

CHAPTER V. CONNECTION BETWEEN SQL AND JAVA

1. SQL Syntax

CREATE: Create databases and tables, a view of a table or other object in the database.

ALTER: Change the structure of an existing database table such as adding a new column, removing column,...

DROP: Delete an existing entire table, a view of a table or other objects in the database.

SELECT: Retrieves certain records from one or more tables.

INSERT: It is used to insert new records in a table.

DELETE: Remove single or multiple in the rows contain data from the database tables depending upon the condition we specify in the WHERE clause.

NOT NULL: Ensures that a column cannot have a NULL value.

UNIQUE: Ensures that all values in a column are different.

PRIMARY KEY: A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table.

FOREIGN KEY: Prevents actions that would destroy links between tables.

WHERE: It is used to filter records/rows.

GROUP BY: It is used to group rows that have the same values in specified columns into aggregated data.

ORDER BY: Data can be sorted in ascending or descending order based on one or more columns.

TOP: It is used to fetch a TOP N number or X percent records from a table.

DISTINCT: It is used to return only distinct (unique) values.

COUNT: It is an SQL aggregate function for counting the number of rows in a particular column.

AND/OR: It is used to filter records based on more than one condition.

JOIN: It is used to combine rows from two or more tables, based on a related column between them.

INNER JOIN: It is used selects records that have matching values in both tables.

2. Relational Algebra, Tree, SQL Query

AdminLogin

This query is used to find the admin details based on the provided username and password.

RA:

$\pi_{Admin_ID, Username, Password} (\sigma_{Username = 'Admin' \text{ and } Password = 'jojo'} (Admin))$

SQL:

```
SELECT * FROM Admin
WHERE Username = 'Admin'
AND Password = 'jojo';
```

ElectionPageAddCandidate

This query is used to fetch the distinct candidate details in a specific election.

RA:

$\pi_{c.Candidate_ID, c.Candidate_No, c.Full_name, c.Gender, c.Age, c.Email} (\sigma_{c.Candidate_ID=v.Candidate_ID \text{ and } vs.Election_ID='SP2024S2'} (\rho_c Candidate \times \rho_v votes))$

SQL:

```
SELECT DISTINCT(c.Candidate_ID), c.Candidate_No, c.Full_name, c.Gender, c.Age, c.Email
FROM Candidate c, votes v
WHERE c.Candidate_ID = v.Candidate_ID
AND v.Election_ID = 'SP2024S2';
```

This query is used to start the time in which the event takes place

RA:

$\pi_{Start_date} (\sigma_{Election_ID = 'SP2024S2'} Election)$

SQL:

```
SELECT Start_date
FROM Election
WHERE Election_ID = 'SP2024S2'
```

This query is used to Count the number of Candidates of the Election

RA:

$\pi_{TotalCandidate} \gamma; COUN(*) \rightarrow TotalCandidate (\sigma_{Election_ID = 'SP2024S2' \text{ and } Voter_ID = null} (votes))$

SQL:

```
SELECT DISTINCT Count(*)
FROM votes
WHERE Election_ID = 'SP2024S2'
And Voter_ID is null;
```

ElectionPage

This query is used to fetch all the election details from the Election table.

RA:

$$\pi_{\text{Election_ID, Election_name, Start_date, End_date}}(\text{Election})$$

SQL:

```
SELECT Election_ID, Election_name, Start_date, End_date FROM Election;
```

ManageVoters

This query is used to fetch all the voter details from the Voter table.

RA:

$$\pi_{\text{Voter_ID, Full_name, Gender, Age, Email, Username, Password}}(\text{Voter})$$

SQL:

```
SELECT Voter_ID, Full_name, Gender, Age, Email, Username, Password  
FROM Voter;
```

Candidates

This query is used to fetch all the candidate details from the Candidate table.

RA:

$$\pi_{\text{Candidate_ID, Full_name, Gender, Age, Email}}(\text{Candidate})$$

SQL:

```
SELECT Candidate_ID, Full_name, Gender, Age, Email  
FROM Candidate;
```

VotersLogin

This query is used to check voter login based on provided username and password

RA:

$$\pi_{\text{Voter_ID}}(\sigma_{\text{Username} = 'pha22161' \text{ and } \text{Password} = '2ltZTtjS'} \text{Voter})$$

SQL:

```
SELECT Voter_ID  
FROM Voter  
WHERE Username = 'pha22161' AND Password = '2ltZTtjS';
```

VotersPage

This query is used to fetch the start date and end date of a specific election based on the provided election ID.

RA:

$$\pi_{Start_date, End_date}(\sigma_{Election_ID = 'SP2024S2'}(Election))$$

SQL:

```
SELECT Start_date, End_date
FROM Election
WHERE Election_ID = 'SP2024S2';
```

This query is used to check if this voter has voted for that election

RA:

$$\pi_{Voter_ID}(\sigma_{Voter_ID = 'ITDSIU22161' \text{ and } Election_ID = 'SP2024S2'}(votes))$$

SQL:

```
SELECT Voter_ID FROM votes
WHERE Voter_ID = 'ITDSIU22161' AND Election_ID = 'SP2024S2'
```

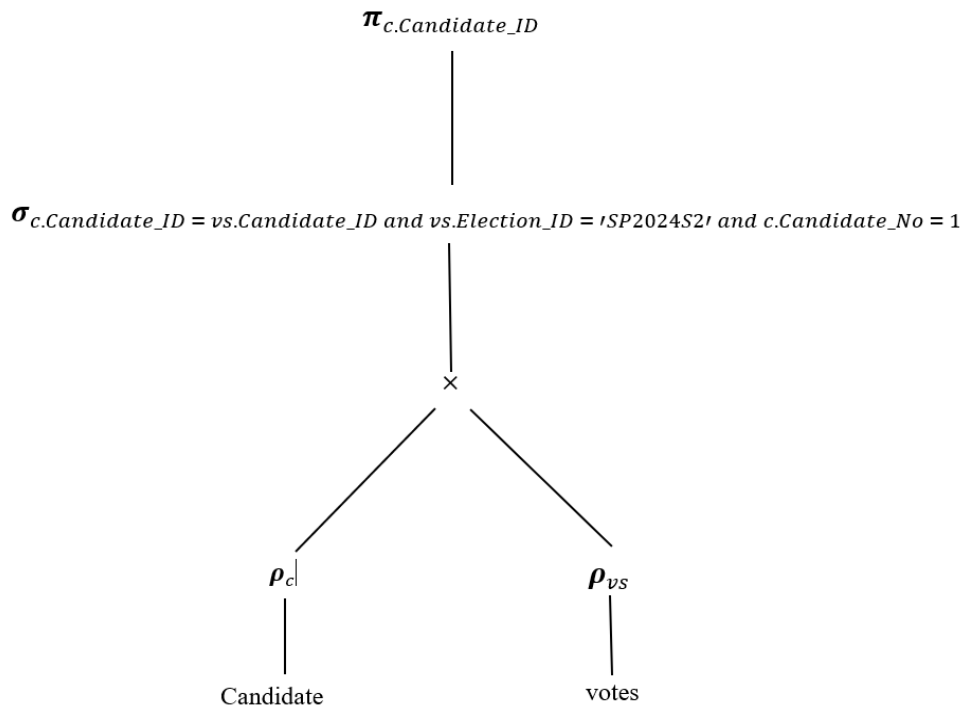
VotersVotingProcess

This query is used to find Candidate_ID of a candidate with candidate_No = 1

RA:

$$\pi_{c.Candidate_ID}(\sigma_{c.Candidate_ID=vs.Candidate_ID \text{ and } vs.Election_ID='SP2024S2' \text{ and } c.Candidate_No = 1}(\rho_c Candidate \times \rho_{vs} votes))$$

Tree:



SQL:

```
Select Distinct c.Candidate_ID
From Candidate as c , votes as vs
Where c.Candidate_ID = vs.Candidate_ID
AND vs.Election_ID = 'SP2024S2'
And Candidate_No =1 ;
```

This query is used to find Candidate_ID of a candidate with candidate_No = 2

RA:

$$\pi_{c.Candidate_ID}(\sigma_{c.Candidate_ID=vs.Candidate_ID \text{ and } vs.Election_ID='SP2024S2' \text{ and } c.Candidate_No = 2}(\rho_c \text{Candidate} \times \rho_{vs} \text{votes}))$$

SQL:

```
Select Distinct c.Candidate_ID
From Candidate as c , votes as vs
Where c.Candidate_ID = vs.Candidate_ID
AND vs.Election_ID = 'SP2024S2'
And Candidate_No = 2;
```

This query is used to find Candidate_ID of a candidate with candidate_No = 3

RA:

$$\pi_{c.Candidate_ID}(\sigma_{c.Candidate_ID=vs.Candidate_ID \text{ and } vs.Election_ID='SP2024S2' \text{ and } c.Candidate_No = 3}(\rho_c \text{Candidate} \times \rho_{vs} \text{votes}))$$

SQL:

```
Select Distinct c.Candidate_ID
From Candidate as c , votes as vs
Where c.Candidate_ID = vs.Candidate_ID
AND vs.Election_ID = 'SP2024S2'
And Candidate_No = 3;
```

This query is used to find Candidate_ID of a candidate with candidate_No = 4

RA:

$$\pi_{c.Candidate_ID}(\sigma_{c.Candidate_ID=vs.Candidate_ID \text{ and } vs.Election_ID='SP2024S2' \text{ and } c.Candidate_No = 4}(\rho_c Candidate \times \rho_{vs} votes))$$

SQL:

```
Select Distinct c.Candidate_ID
From Candidate as c , votes as vs
Where c.Candidate_ID = vs.Candidate_ID
AND vs.Election_ID = 'SP2024S2'
And Candidate_No = 4;
```

This query is used to find Candidate_ID of a candidate with candidate_No = 5

RA:

$$\pi_{c.Candidate_ID}(\sigma_{c.Candidate_ID=vs.Candidate_ID \text{ and } vs.Election_ID='SP2024S2' \text{ and } c.Candidate_No = 5}(\rho_c Candidate \times \rho_{vs} votes))$$

SQL:

```
Select Distinct c.Candidate_ID
From Candidate as c , votes as vs
Where c.Candidate_ID = vs.Candidate_ID
AND vs.Election_ID = 'SP2024S2'
And Candidate_No = 5;
```

ElectionResults

This query is used to count the number of voters who voted

RA:

$$\pi_{Voter_Count} \gamma; COUNT(v.Voter_ID) \rightarrow Voter_Count \sigma_{vs.Election_ID = 'SP2024S2' \text{ and } v.Voter_ID \text{ null} \neq \text{null}} \rho_v votes$$

SQL:

```
Select count(v.Voter_ID) as Voter_Count
From votes as v
Where v.Election_ID = 'SP2024S2'
And v.Voter_ID is not null
```

This query is used to count the number of participating candidates

RA:

$$\gamma_{COUNT(DISTINCT Candidate_ID)} \rightarrow total (\pi_{Candidate_ID} (\sigma_{Election_ID = 'SP2024S2'}(votes)))$$

SQL:

```
Select count(v.Voter_ID) as Voter_Count
From votes as v
Where v.Election_ID = 'SP2024S2'
And v.Voter_ID is not null
```

This query is used to find candidate names and the percentage of votes the candidate who has Candidate_No = 1.

RA:

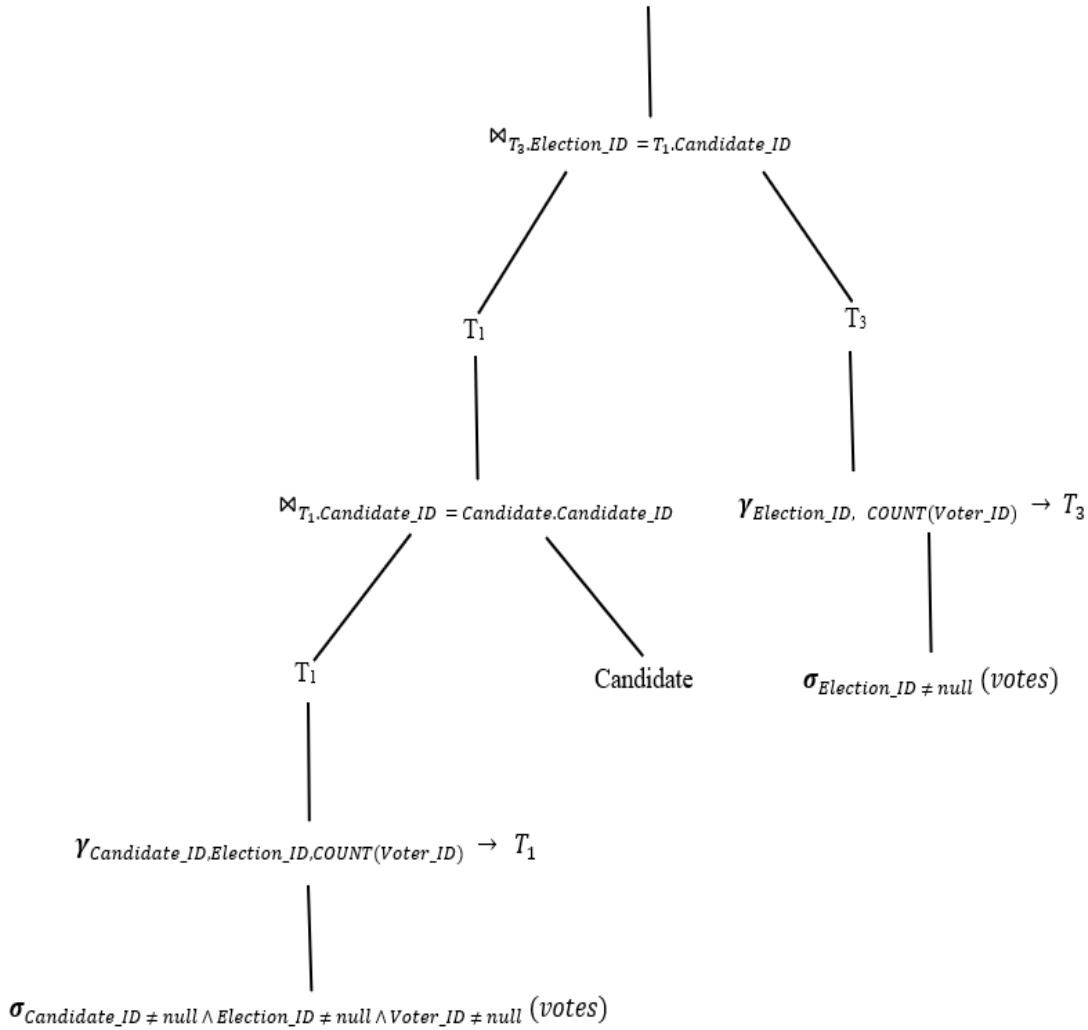
$$\begin{aligned} total_votes_base_candidate (T_1) &\leftarrow \gamma_{Candidate_ID, Election_ID, COUNT(Voter_ID)} \\ &\rightarrow total_base_on_voter (T_2) (\sigma_{Candidate_ID \neq null \text{ and } Election_ID \neq null \text{ and } Voter_ID \neq null} (votes)) \\ total_base_on_election(T_3) &\leftarrow \gamma_{Election_ID, COUNT(Voter_ID)} \rightarrow T_2(\sigma_{Election_ID \neq null} (votes)) \end{aligned}$$

$$\pi_{Candidate.Candidate_ID, Candidate.Candidate_No, Candidate.Full_name, T_3.Election_ID, Percentage} (\sigma_{Candidate.Candidate_No = 1 \wedge T_3.Election_ID = 'SP2024S2'})$$

$$(T_3 \bowtie_{T_3.Election_ID = T_1.Election_ID} T_1 \bowtie_{T_1.Candidate_ID = Candidate.Candidate_ID} Candidate) \mid$$

Tree:

$\pi_{Candidate.Candidate_ID, Candidate.Candidate_No, Candidate.Full_name, T_3.Election_ID, Percentage} (\sigma_{Candidate.Candidate_No = 1 \wedge T_3.Election_ID = 'SP2024S2'})$



SQL:

```
With total_votes_base_candidate as (
  Select Candidate_ID, Election_ID, COUNT(Voter_ID) 'total based on candidate'
  From votes
  Where Candidate_ID is not null
  And Election_ID is not null
  And Voter_ID is not null
  Group by Candidate_ID, Election_ID
),
total_based_on_election as (
  Select Election_ID, count(Voter_ID) 'total based on Voter'
  From votes
  Where Election_ID is not null
  Group by Election_ID
)
Select Candidate.Candidate_ID,
       Candidate.Candidate_No,
       Candidate.Full_name,
       total_based_on_election.Election_ID,
       Cast(Cast([total based on candidate]*100.0 / [total based on Voter]) as decimal(18,2)) as varchar) + '%' PERCENTAGE
From total_based_on_election, total_votes_base_candidate, Candidate
Where total_based_on_election.Election_ID = total_votes_base_candidate.Election_ID
And Candidate.Candidate_ID = [total_votes_base_candidate].Candidate_ID
And Candidate_No = 1
And total_based_on_election.Election_ID = 'SP2024S2'
```

This query is used to find candidate names and the percentage of votes the candidate who has Candidate_No = 2.

RA:

$$\begin{aligned} total_votes_base_candidate(T_1) &\leftarrow \gamma_{Candidate_ID, Election_ID, COUNT(Voter_ID)} \\ &\rightarrow total_base_on_voter(T_2) (\sigma_{Candidate_ID \neq null \text{ and } Election_ID \neq null \text{ and } Voter_ID \neq null} (votes)) \\ total_base_on_election(T_3) &\leftarrow \gamma_{Election_ID, COUNT(Voter_ID)} \rightarrow T_2(\sigma_{Election_ID \neq null} (votes)) \end{aligned}$$

$$\pi_{Candidate.Candidate_ID, Candidate.Candidate_No, Candidate.Full_name, T_3.Election_ID, Percentage} (\sigma_{Candidate.Candidate_No = 2 \wedge T_3.Election_ID = 'SP2024S2'})$$

$$(T_3 \bowtie_{T_3.Election_ID = T_1.Election_ID} T_1 \bowtie_{T_1.Candidate_ID = Candidate.Candidate_ID} Candidate)$$

SQL:

```
With total_votes_base_candidate as (
Select Candidate_ID, Election_ID, COUNT(Voter_ID) 'total based on candidate'
From votes
Where Candidate_ID is not null
And Election_ID is not null
And Voter_ID is not null
Group by Candidate_ID, Election_ID
),
total_based_on_election as (
Select Election_ID, count(Voter_ID) 'total based on Voter'
From votes
Where Election_ID is not null
Group by Election_ID
)
Select Candidate.Candidate_ID,
Candidate.Candidate_No,
Candidate.Full_name,
total_based_on_election.Election_ID,
Cast(Cast([total based on candidate]*100.0 / [total based on Voter] as decimal(18,2)) as varchar) + '%' PERCENTAGE
From total_based_on_election, total_votes_base_candidate, Candidate
Where total_based_on_election.Election_ID = total_votes_base_candidate.Election_ID
And Candidate.Candidate_ID = [total_votes_base_candidate].Candidate_ID
And Candidate_No = 2
And total_based_on_election.Election_ID = 'SP2024S2'
```

This query is used to find candidate names and the percentage of votes the candidate who has Candidate_No = 3.

RA:

$$\begin{aligned} total_votes_base_candidate(T_1) &\leftarrow \gamma_{Candidate_ID, Election_ID, COUNT(Voter_ID)} \\ &\rightarrow total_base_on_voter(T_2) (\sigma_{Candidate_ID \neq null \text{ and } Election_ID \neq null \text{ and } Voter_ID \neq null} (votes)) \\ total_base_on_election(T_3) &\leftarrow \gamma_{Election_ID, COUNT(Voter_ID)} \rightarrow T_2(\sigma_{Election_ID \neq null} (votes)) \end{aligned}$$

$$\pi_{Candidate.Candidate_ID, Candidate.Candidate_No, Candidate.Full_name, T_3.Election_ID, Percentage} (\sigma_{Candidate.Candidate_No = 3 \wedge T_3.Election_ID = 'SP2024S2'})$$

$$(T_3 \bowtie_{T_3.Election_ID = T_1.Election_ID} T_1 \bowtie_{T_1.Candidate_ID = Candidate.Candidate_ID} Candidate)$$

SQL:

```

With total_votes_base_candidate as (
Select Candidate_ID, Election_ID, COUNT(Voter_ID) 'total based on candidate'
From votes
Where Candidate_ID is not null
And Election_ID is not null
And Voter_ID is not null
Group by Candidate_ID, Election_ID
),
total_based_on_election as (
Select Election_ID, count(Voter_ID) 'total based on Voter'
From votes
Where Election_ID is not null
Group by Election_ID
)
Select Candidate.Candidate_ID,
Candidate.Candidate_No,
Candidate.Full_name,
total_based_on_election.Election_ID,
Cast(Cast([total based on candidate]*100.0 / [total based on Voter]) as decimal(18,2)) as varchar) + '%' PERCENTAGE
From total_based_on_election, total_votes_base_candidate, Candidate
Where total_based_on_election.Election_ID = total_votes_base_candidate.Election_ID
And Candidate.Candidate_ID = [total_votes_base_candidate].Candidate_ID
And Candidate_No = 3
And total_based_on_election.Election_ID = 'SP2024S2'

```

This query is used to find candidate names and the percentage of votes the candidate who has Candidate_No = 4.

RA:

$$\begin{aligned}
total_votes_base_candidate(T_1) &\leftarrow \gamma_{Candidate_ID, Election_ID, COUNT(Voter_ID)} \\
&\rightarrow total_base_on_voter(T_2) (\sigma_{Candidate_ID \neq null \text{ and } Election_ID \neq null \text{ and } Voter_ID \neq null} (votes)) \\
total_base_on_election(T_3) &\leftarrow \gamma_{Election_ID, COUNT(Voter_ID)} \rightarrow T_2(\sigma_{Election_ID \neq null} (votes))
\end{aligned}$$

$$\pi_{Candidate.Candidate_ID, Candidate.Candidate_No, Candidate.Full_name, T_3.Election_ID, Percentage} (\sigma_{Candidate.Candidate_No = 4 \wedge T_3.Election_ID = 'SP2024S2'})$$

$$(T_3 \bowtie_{T_3.Election_ID = T_1.Election_ID} T_1 \bowtie_{T_1.Candidate_ID = Candidate.Candidate_ID} Candidate$$

SQL:

```

With total_votes_base_candidate as (
Select Candidate_ID, Election_ID, COUNT(Voter_ID) 'total based on candidate'
From votes
Where Candidate_ID is not null
And Election_ID is not null
And Voter_ID is not null
Group by Candidate_ID, Election_ID
),
total_based_on_election as (
Select Election_ID, count(Voter_ID) 'total based on Voter'
From votes
Where Election_ID is not null
Group by Election_ID
)
Select Candidate.Candidate_ID,
Candidate.Candidate_No,
Candidate.Full_name,
total_based_on_election.Election_ID,
Cast(Cast([total based on candidate]*100.0 / [total based on Voter]) as decimal(18,2)) as varchar) + '%' PERCENTAGE
From total_based_on_election, total_votes_base_candidate, Candidate
Where total_based_on_election.Election_ID = total_votes_base_candidate.Election_ID
And Candidate.Candidate_ID = [total_votes_base_candidate].Candidate_ID
And Candidate_No = 4
And total_based_on_election.Election_ID = 'SP2024S2'

```

This query is used to find candidate names and the percentage of votes the candidate who has Candidate_No = 5.

RA:

$$\begin{aligned} total_votes_base_candidate (T_1) &\leftarrow \gamma_{Candidate_ID, Election_ID, COUNT(Voter_ID)} \\ &\rightarrow total_base_on_voter (T_2) (\sigma_{Candidate_ID \neq null \text{ and } Election_ID \neq null \text{ and } Voter_ID \neq null} (votes)) \\ total_base_on_election(T_3) &\leftarrow \gamma_{Election_ID, COUNT(Voter_ID)} \rightarrow T_2(\sigma_{Election_ID \neq null} (votes)) \end{aligned}$$

$$\pi_{Candidate.Candidate_ID, Candidate.Candidate_No, Candidate.Full_name, T_3.Election_ID, Percentage} (\sigma_{Candidate.Candidate_No = 5 \wedge T_3.Election_ID = 'SP2024S2'})$$

$$(T_3 \bowtie_{T_3.Election_ID = T_1.Election_ID} T_1 \bowtie_{T_1.Candidate_ID = Candidate.Candidate_ID} Candidate$$

SQL:

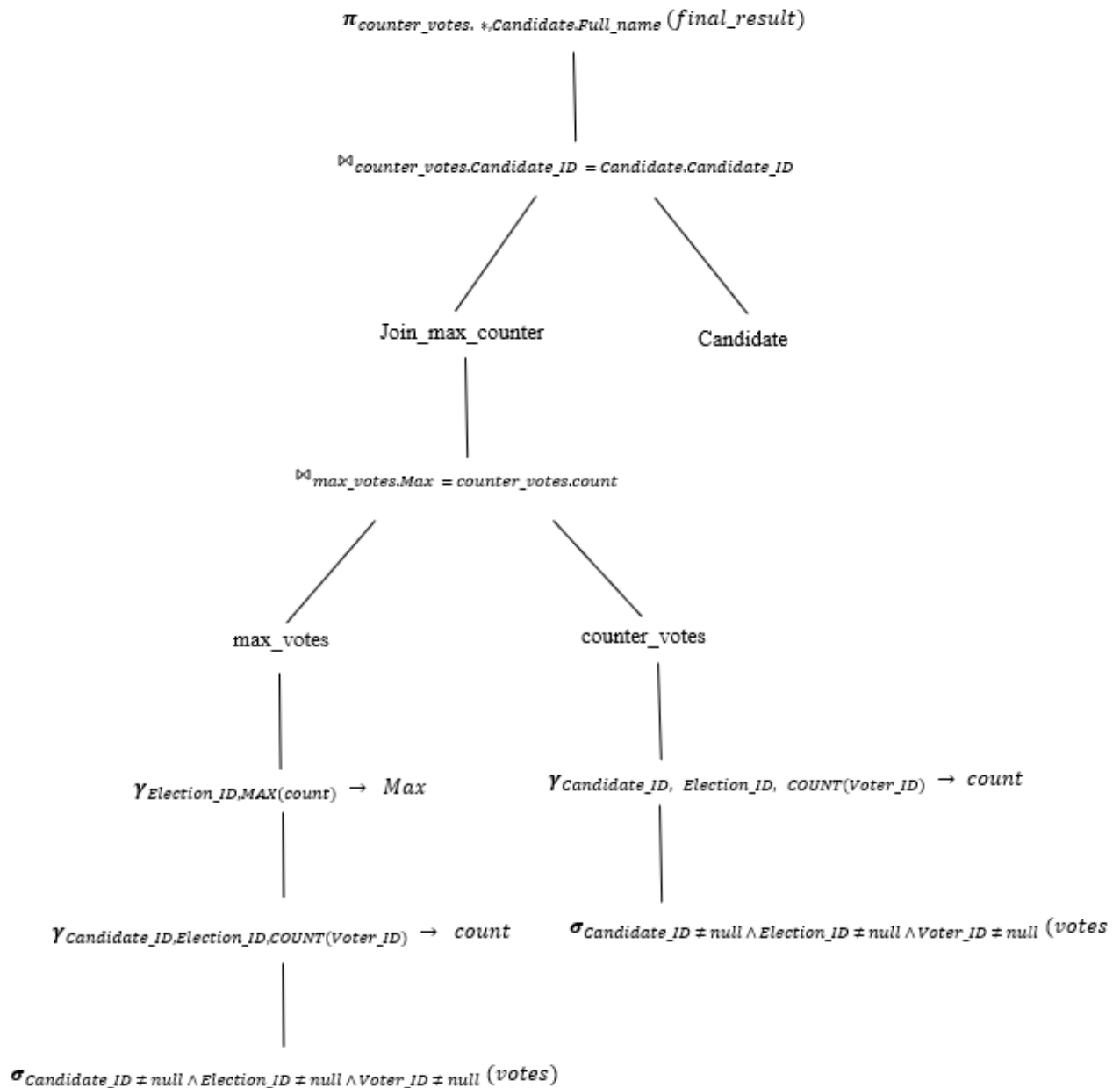
```
With total_votes_base_candidate as (
Select Candidate_ID, Election_ID, COUNT(Voter_ID) 'total based on candidate'
From votes
Where Candidate_ID is not null
And Election_ID is not null
And Voter_ID is not null
Group by Candidate_ID, Election_ID
),
total_based_on_election as (
Select Election_ID, count(Voter_ID) 'total based on Voter'
From votes
Where Election_ID is not null
Group by Election_ID
)
Select Candidate.Candidate_ID,
Candidate.Candidate_No,
Candidate.Full_name,
total_based_on_election.Election_ID,
Cast(Cast([total based on candidate]*100.0 / [total based on Voter]) as decimal(18,2)) as varchar) + '%' PERCENTAGE
From total_based_on_election, total_votes_base_candidate, Candidate
Where total_based_on_election.Election_ID = total_votes_base_candidate.Election_ID
And Candidate.Candidate_ID = [total_votes_base_candidate].Candidate_ID
And Candidate_No = 5
And total_based_on_election.Election_ID = 'SP2024S2'
```

This query is used to find determine the candidates who received the highest number of votes in each election.

RA:

$$\begin{aligned} counter_votes (T_1) &\leftarrow \gamma_{Candidate_ID, Election_ID, COUNT(Voter_ID)} \\ &\rightarrow count (\sigma_{Candidate_ID \neq null \text{ and } Election_ID \neq null \text{ and } Voter_ID \neq null} (votes)) \\ max_votes (T_2) &\leftarrow \gamma_{Election_ID, MAX(count)} \rightarrow Max (\gamma_{Candidate_ID, Election_ID, COUNT(Voter_ID)} \\ &\rightarrow count (\sigma_{Candidate_ID \neq null \text{ and } Election_ID \neq null \text{ and } Voter_ID \neq null} (votes))) \\ Join_max_counter (T_3) &\leftarrow T_2 \bowtie_{T_2.Max = T_1.count} T_1 \\ Final_result (T_4) &\leftarrow T_3 \bowtie_{T_3.Candidate_ID = candidate.Candidate_ID} Candidate \\ &\pi_{T_1.*, Candidate.Full_name} (T_4) \end{aligned}$$

Tree:



SQL:

```

With counter_votes as (
  Select Candidate_ID, Election_ID, Count(Voter_ID) 'count'
  From votes
  Where Candidate_ID is not null and
  Election_ID is not null
  And Voter_ID is not null
  Group by votes.Candidate_ID, Election_ID),
max_votes as (Select max(count) 'Max'
               From counter_votes
               Group by Election_ID)
Select counter_votes.*, Candidate.Full_name
From max_votes, counter_votes, Candidate
Where max_votes.[Max] = counter_votes.[count]
And counter_votes.Candidate_ID = Candidate.Candidate_ID

```

3. Application forms

In our online voting system, the application form serves as the graphical user interface (GUI) that allows users to interact with the SQL database through our Java application. This form collects user inputs, processes them, and performs database operations such as inserting, updating, or retrieving data. Here's a detailed breakdown of how we create an application form to connect Java to SQL.

3.1 Set Up Development Environment

We ensure that the Java Development Kit (JDK) is installed and uses an Integrated Development Environment (IDE) like IntelliJ IDEA, Eclipse, or NetBeans. We also set up a SQL database like SQL Server.

3.2 Create Database Schema

We define the structure of our database tables. For example, for the voter registration form, we might have a `Voters` table.

3.3 Database Connection in Java and Create the Application Form Using Swing

- We write a Java class to establish a connection to the SQL database using JDBC (Java Database Connectivity).

- After that, we use Swing, part of Java's standard library, is used to create the GUI. Below is a simple example of a voter login form:

- Creates a GUI form using Swing with fields for username and password and a login button.
- When the login button is clicked, the loginUser method is called, which retrieves the input from the form fields.
- Establishes a connection to the database using `DatabaseConnection.getConnection()`.
- Checks the user credentials in the voters table using a prepared statement to prevent SQL injection.
- Displays a success or error message based on the result of the operation.

```

private void jButton12ActionPerformed(java.awt.event.ActionEvent evt) {
    //GEN-FIRST:event_jButton12ActionPerformed
    String username = jTextField2.getText();
    String password = new String(jPasswordField1.getPassword());
    // Convert password to string correctly
    System.out.println("pwd: " + password);

    if(username.isEmpty() || password.isEmpty()) {
        JOptionPane.showMessageDialog(this, message:"Username / Password should not be empty.");
    } else {
        String serverName = "DESKTOP-0IKLTBG";
        String databaseName = "Online-Voting";
        String url = "jdbc:sqlserver://" + serverName
            + ":1433;databaseName="
            + databaseName
            + ";encrypt=true;trustServerCertificate=true;";

        try {
            con = DriverManager.getConnection(url, user:"sa", password:"123456789");
            String query = "SELECT Voter_ID FROM Voter WHERE Username = ? AND Password = ?";
            pst = con.prepareStatement(query);
            pst.setString(parameterIndex:1, jTextField2.getText());
            pst.setString(parameterIndex:2, jPasswordField1.getText());
            rs = pst.executeQuery();

            if(rs.next()) {
                String Voter_ID = rs.getString(columnLabel:"Voter_ID");
                JOptionPane.showMessageDialog(this, message:"Login Successful");
                VotersPage v = new VotersPage(Voter_ID);
                v.setVisible(b:true);
                dispose();
            } else {
                JOptionPane.showMessageDialog(this, message:"Login Failed");
            }
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(parentComponent:null, ex);
        }
    }
}
//GEN-LAST:event_jButton12ActionPerformed

```

Figure 4.3.1 Voter login code

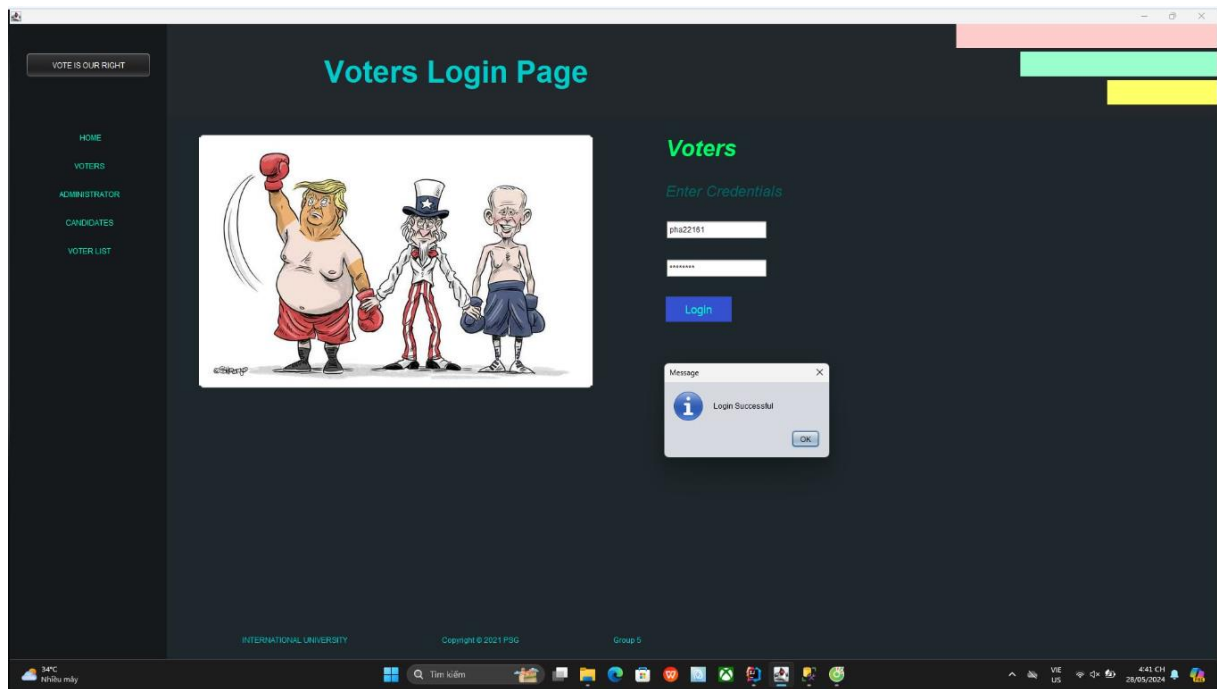


Figure 4.3.2 Successful Voter login

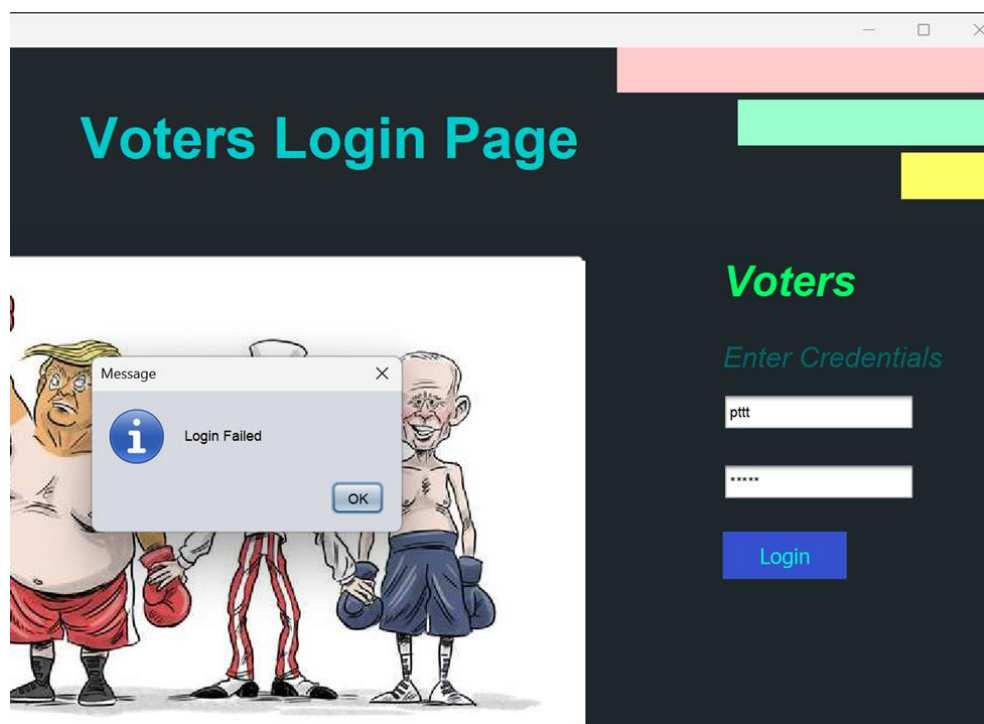


Figure 4.3.3 Failed Voter login

In our online voting system, the application form is a GUI-based interface that interacts with the SQL database to perform various operations like voter login. The provided example demonstrates a simple voter login form, but the same principles can be applied to create more complex forms and interactions.

4. Demo

There are main pages in our application:

4.1 Admin

1.1 Admin login

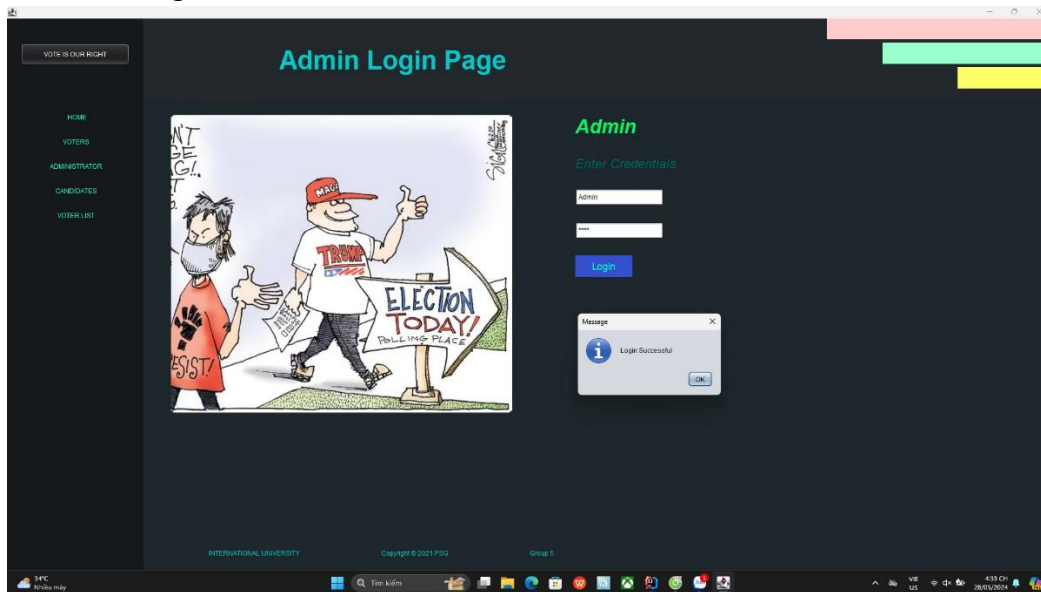


Figure 4.4.1 Admin sign-up interface

Administrators can access the Admin Page by entering their correct Admin ID and password, and then checking the "Log In" box to complete the process.

1.2 Admin Page

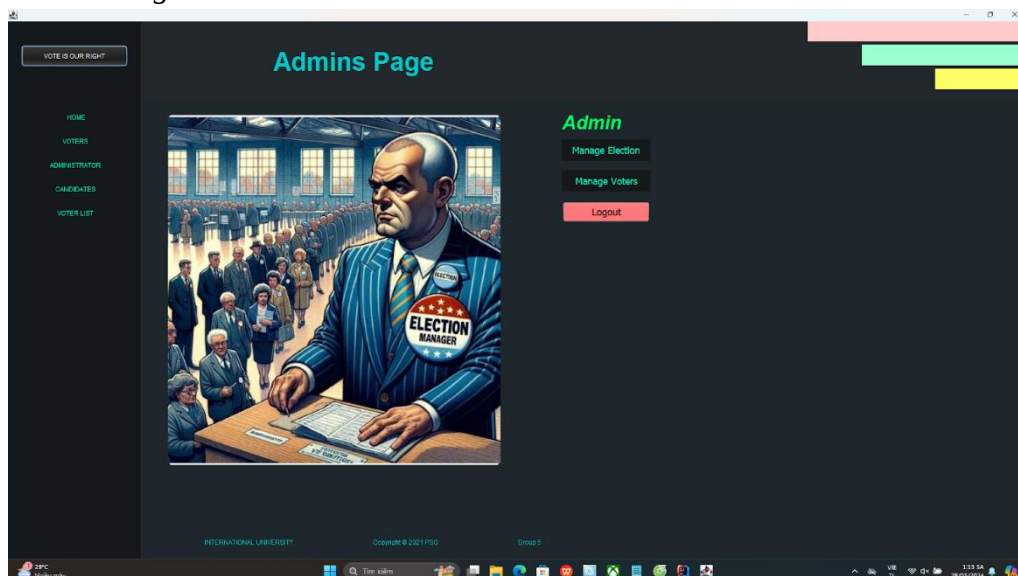


Figure 4.4.2 Admin manage Voters

The Admin Page empowers administrators to create elections ("Create Election" box) and manage voter information ("Manage Voter" box).

1.3 Create Election Page

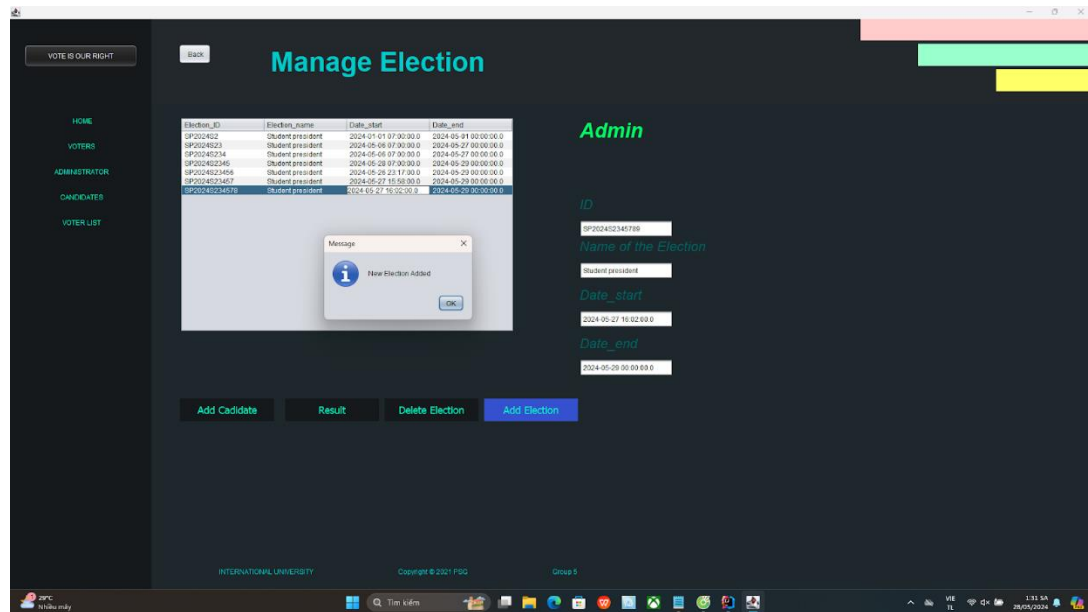


Figure 4.4.3 Add Election

On this page, administrators can manage elections. To add a new election, they need to fill in the required information: Election ID, Name, Start Date, and End Date. After filling in the details, they must check the "Add Election" box to finalize the process. To delete an existing election, they simply need to select the election they want to delete and then click the "Delete Election" box.

Once an election is created, administrators can add new candidates by checking the "Add Candidate" box. This action will redirect them to the "Add Candidate Page".

1.3.1 Add Candidate Page

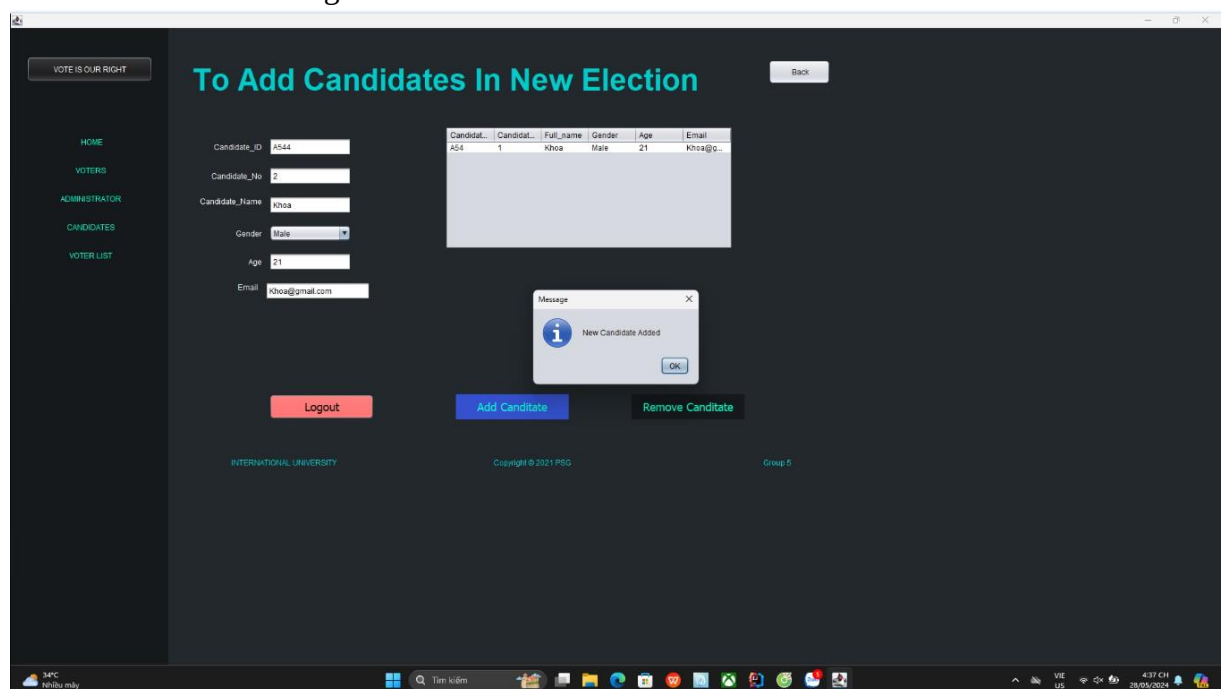


Figure 4.4.4 Add Candidate

The Add Candidate Page allows administrators to input candidate information into text fields. After filling in the details, they need to check the "Add Candidate" box. To delete a candidate, administrators can select the candidate's entry in the table and check the "Remove Candidate" box.

NOTE: Once the election's start date arrives, administrators lose the ability to add or modify candidate information. This ensures the integrity of the election process and prevents any last-minute changes that could impact the fairness of the voting.

1.3.2 Manage Voters Page

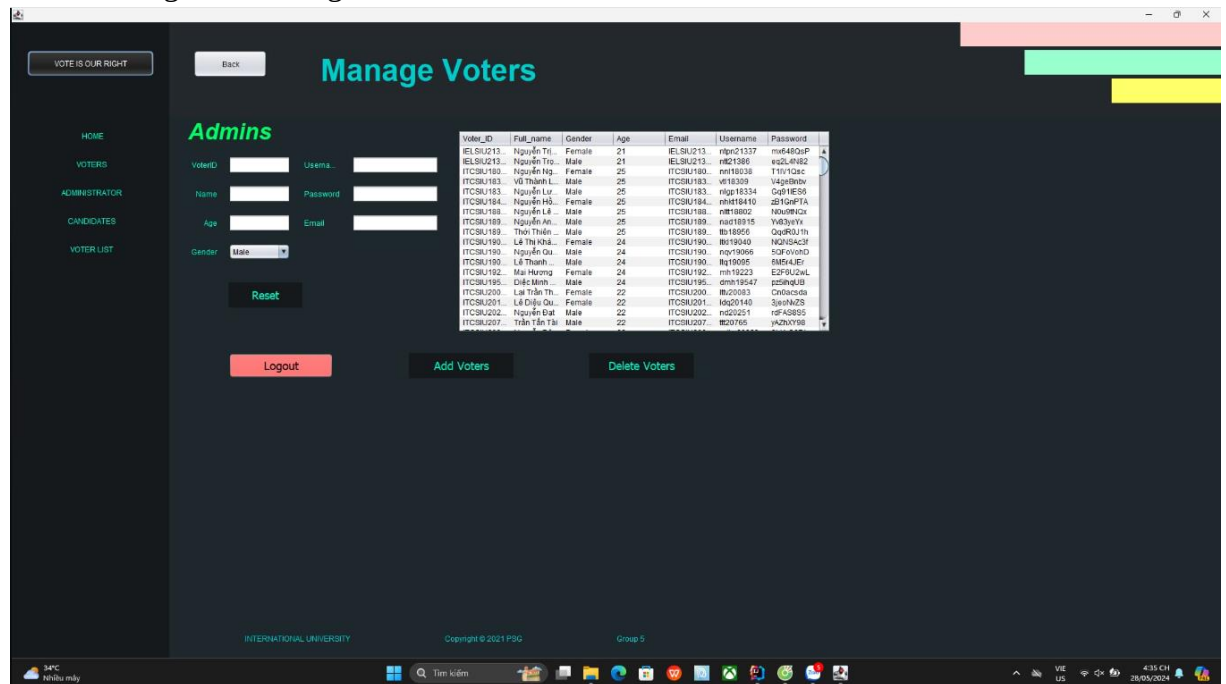


Figure 4.4.5 Manage Voters

On the Manage Voters Page, administrators can add new voters by filling in their information and checking the "Add Voter" box. To remove a voter, they can simply select the voter's entry from the table and check the "Delete Voters" box.

4.2 Voter

2.1 Voter login

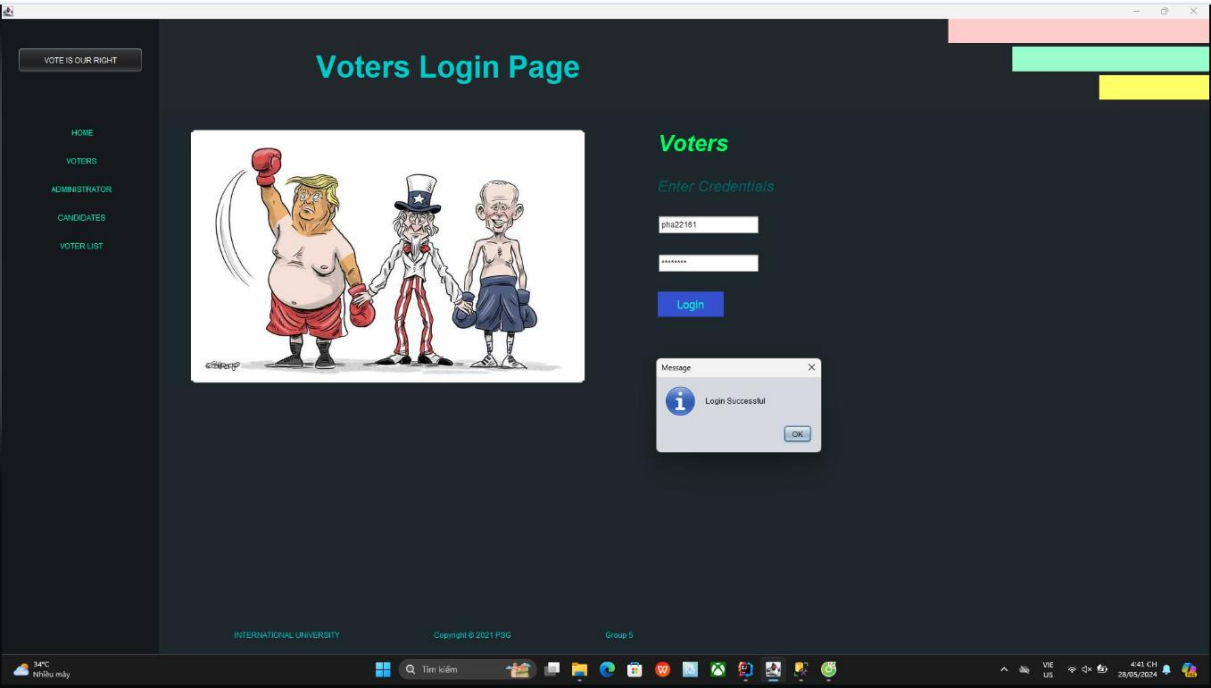


Figure 4.4.6 Voter sign-up interface

Voters can access the Voter Page by entering their unique Voter ID and password, provided by the administrator, and then checking the "Log In" box.

2.2 Voter Page

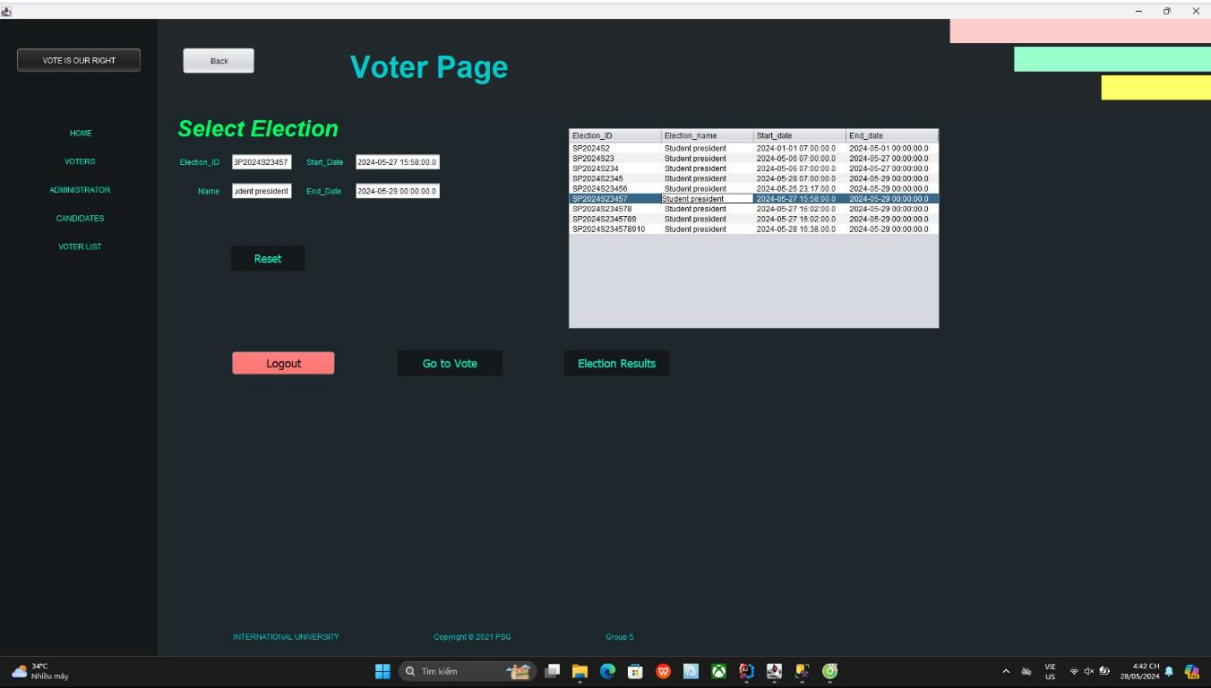


Figure 4.4.7 Voter select Election

On the Voter Page, voters can select an election from the table to proceed to the voting process. However, if the election hasn't started yet or has already ended, voters will not be able to access the voting processing.

Once a voter has chosen an election from the table, they can proceed to the Vote Page by checking the "Go to Vote" box.

2.3 Vote Page

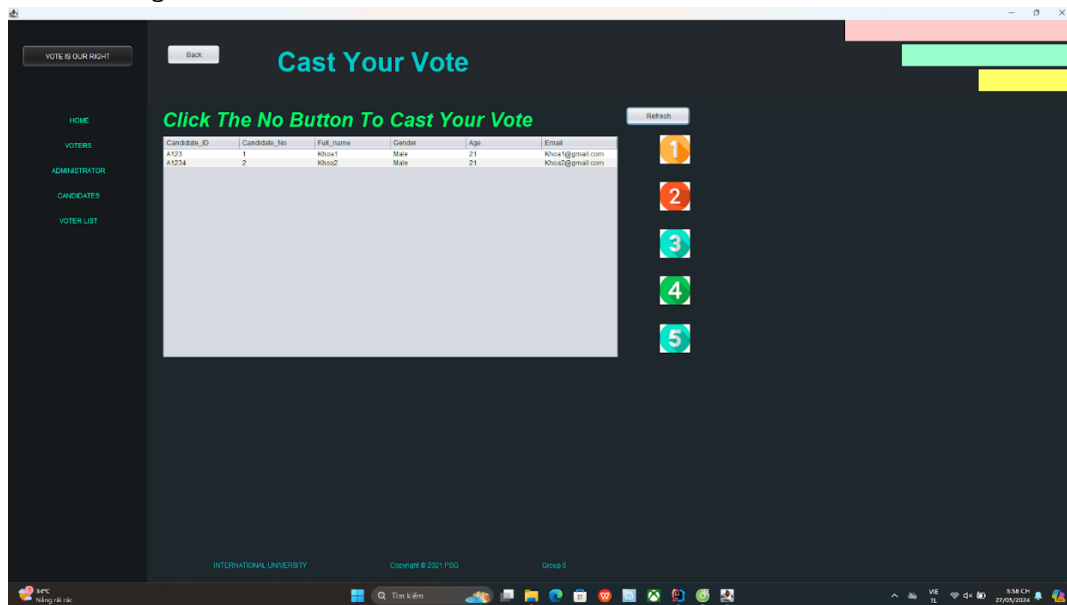


Figure 4.4.8 Cast your vote

On the Vote Page, voters should refresh the candidate information to see the latest voting status. They can then cast their vote by selecting the candidate's number, which corresponds to the number provided next to their ID and name.

4.3 Election Result

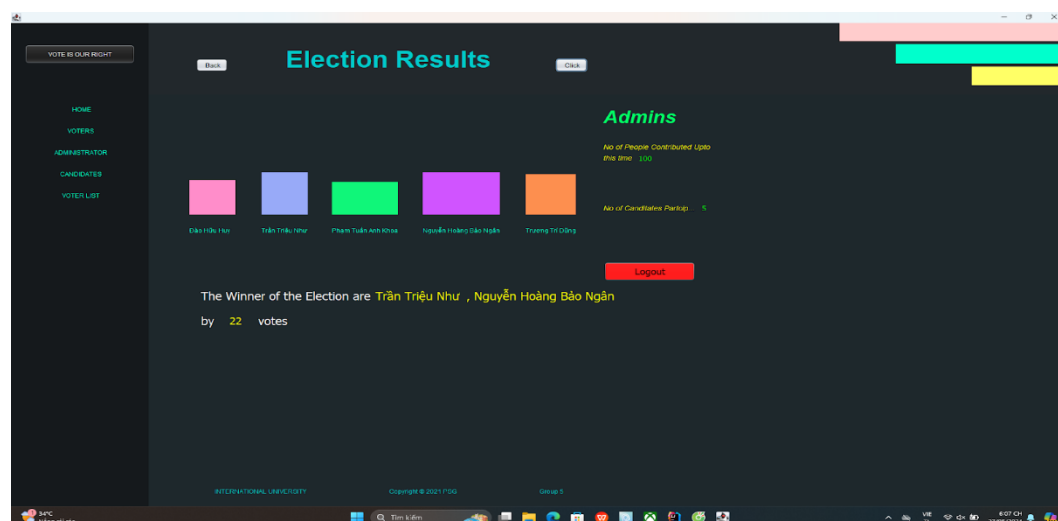


Figure 4.4.9 Election results

3.1 For Admin

Administrators can monitor the voting process in real time by checking the "Election Results" option on the Create Election Page. This allows them to track the progress of the vote count as it unfolds.

3.2 For Voters

Voters can access “Election Results” on the Voter Page, but only after the election has concluded. They will not be able to view results during the voting process.

CHAPTER VI. CONCLUSION

1. Conclusion

Our project includes a user application featuring a UI designed for voting. This application supports two main objects. Firstly, administrators play pivotal roles in the online voting system with responsibilities including creating elections and managing voters. Administrators have the authority to create elections, dictate their timing, and control crucial aspects. Supervisors can oversee the election process but cannot add or remove candidates while the election is in progress. Secondly, users utilize the application to log in and participate in elections, provided their names are on the approved voter list, adhering to the "one person, one vote, one election" principle. Voters only see the results once the election concludes.

Nevertheless, there are several shortcomings that we will address going forward. Firstly, the system hasn't integrated authorization and currently, the system does not distinguish roles into candidates and voters. As a result, if duplicate candidates are not properly managed, this issue could result in the same candidate winning multiple elections. Secondly, the system currently restricts voter and candidate registration, allowing only developers to create data for testing purposes. Addressing these issues will significantly enhance the system's functionality and user experience.

2. Recommendation

Integrate Authorization and Distinguish Roles:

We will ensure the system distinguishes between candidates and voters to prevent unauthorized actions. Clear roles will be set for candidates and voters, specifying their permissions. Thorough testing will guarantee proper handling of these roles.

Open Registration for Voters and Candidates:

Public registration will be extended beyond developers. This entails creating user-friendly sign-up forms for voters and candidates, along with implementing email verification to authenticate registrations.

Upgrade to a Modern Interface:

The online voting application's interface will be enhanced to ensure it is user-friendly, visually appealing, and responsive across different devices. This may include implementing a notification system for real-time updates and providing a comprehensive help menu with various support tools and resources integrated within the app interface.

3. What we learn after project ?

The members of our group have gained a lot of knowledge after finishing this project, including:

- Relational model ERD and Schema
- Relational Algebra SQL.
- Create a database.
- Insert data to a database.
- Make a query SQL Server.
- Connect Java Display form and SQL Server

Reference:

- [1] What is an entity relationship diagram (ERD)? Available at: <https://www.lucidchart.com/pages/er-diagrams>
- [2] Enhanced ER diagram tutorial. Available at: <https://www.lucidchart.com/pages/enhanced-entity-relationship-diagram>
- [3] 'Lecture 3: ERD translation', Principles of Database Management. Principles of Database Management, Thủ Đức: International University – HCMIU, April.
- [4] Hoffer, J., Ramesh, V. and Topi, H. (2016) Modern Database Management. Harlow, United Kingdom: Pearson Education Limited.
- [5] Connect SQL server with Visual Studio 2022. Available at: <https://www.youtube.com/watch?v=HeGNMgdQ79M>
- [6] Connect to SQL server in Java. Available at: <https://blog.devart.com/connect-to-sql-server-in-java.html>
- [7] What is the Boyce-Codd Normal Form (BCNF)? - definition from Techopedia, Techopedia. Available at: <https://www.techopedia.com/definition/5642/boyce-codd-normal-form-bcnf>
- [7] Bài giảng cơ sở dữ liệu: 06 - Chuyển đổi liên kết cấp 3 thành các liên kết cấp 2 trong mô hình ER
YouTube. Available at: <https://www.youtube.com/watch?v=WXGRgE8z9dI>
- [8] GitHub. Available at: https://github.com/Sarath-Ra/Voting-System-Desktop-Application-?fbclid=IwZXh0bgNhZW0CMTAAR02y6M--h4_qTIE71jc6CcS_Ipgam2Nxx77eRfTjBWw2h_7H8lJfrfzu5w_aem_ZmFrZW1bW15MTZieX
- [9] Kết nối IntelliJ với SQL Server. Available at: <https://www.youtube.com/watch?v=RF-vchtV58&t=2s>
- [10] A. Silberschatz, H. F. Korth, S. Sudarshan, Database system concepts, 7th edition, McGraw Hill Education, 2011, pp. 283–284. Available at: <https://db-book.com/slides-dir/PDF-dir/ch6.pdf>
- [11] SQL Syntax. Available at: <https://www.w3schools.com/sql/>
- [12] How to Install Microsoft SQL Server & SSMS on Windows 10/11 [2023 Update] Complete guide. Available at: https://www.youtube.com/watch?v=iaUXjTL_F9U